

RAPPORT DE PROJET TECHNIQUE

*Orchestration et Conteneurisation d'une Application de CPU Scheduling
avec Docker Compose et MySQL*

Réalisé par : *sidelemine elghadi
ahmed bechir
tfeylouhe ahmedou*

Encadré par : *Dr. EL BENANY*

Table des Matières

1. Introduction Générale
2. Contexte du Projet et Objectifs
3. Architecture Technique du Système
4. Développement de l'Application Éducative (Python/Flask)
5. Configuration de la Conteneurisation (Dockerfile & Docker Compose)
6. Déploiement, Cycle de Vie et Exécution du Projet
7. Validation des Résultats et Tests d'Intégration
8. Matrice de Contribution et Partage des Tâches
9. Conclusion Générale

1. Introduction Générale

Dans le domaine de l'ingénierie logicielle contemporaine, l'évolution vers des architectures découplées et modularisées a transformé les méthodologies de déploiement. La technologie de conteneurisation, menée par la plateforme open-source Docker, s'est imposée comme un standard de l'industrie. Elle permet d'encapsuler une application et ses dépendances dans un environnement isolé, résolvant ainsi définitivement le problème classique du « ça fonctionne sur ma machine ».

Ce rapport technique présente la conception, le développement et l'orchestration multi-conteneurs d'une application de simulation des algorithmes d'ordonnancement du processeur (CPU Scheduling). Ce projet combine les principes fondamentaux des systèmes d'exploitation avec les pratiques modernes d'administration système et de virtualisation légère sous l'environnement Windows Subsystem for Linux (WSL 2).

2. Contexte du Projet et Objectifs

L'ordonnancement du processeur émane des mécanismes internes par lesquels un système d'exploitation alloue le temps CPU aux différents processus en attente d'exécution. Pour comprendre ces mécanismes abstraits, nous avons développé un outil de simulation visuel. Cependant, l'accent principal de ce travail est mis sur l'infrastructure de déploiement.

2.1 Objectifs Fonctionnels

- **Simulation Algorithmique** : Implémentation fonctionnelle des algorithmes d'ordonnancement fondamentaux, notamment le FIFO (First-In, First-Out) et le SRTF (Shortest Remaining Time First).
- **Persistance des Données** : Sauvegarde systématique des configurations de processus injectées et des métriques de performance calculées (temps d'attente, temps de rotation) dans un système de gestion de base de données relationnelle.

2.2 Objectifs Techniques et Infrastructure

- **Micro-services** : Séparation stricte de la logique applicative (Web Flask) et du stockage (Moteur MySQL) dans deux environnements isolés.
- **Isolation Réseau** : Création d'un réseau virtuel privé Docker permettant une communication bidirectionnelle sécurisée entre les conteneurs sans exposer inutilement les ports sensibles sur la machine hôte.
- **Automated Deployment** : Centralisation de la configuration à l'aide d'un orchestrateur local (Docker Compose) pour permettre un déploiement instantané via une commande unique.

3. Architecture Technique du Système

L'architecture globale de notre solution repose sur le patron de conception micro-services, caractérisé par deux composants autonomes qui collaborent de manière transparente à l'intérieur d'un espace réseau virtualisé.

3.1 Description des Composants de l'Infrastructure

Composant (Service)	Image de Base Appliquée	Rôle au Sein de l'Architecture	Port Interne / Externe
web_app	python:3.9-slim	Exécute le serveur Flask, traite l'algorithme d'ordonnancement et expose l'interface utilisateur.	5000 : 5000
database	mysql:8.0	Héberge le serveur relationnel MySQL et stocke les historiques d'exécution.	3306 : Aucun (Isolé)

3.2 Gestion de l'Interconnexion Réseau

Par défaut, Docker Compose configure un réseau virtuel spécifique pour l'application. Le conteneur `web_app` utilise le résolveur DNS interne de Docker pour contacter le service de base de données en utilisant simplement le nom d'hôte `database`, éliminant ainsi le besoin de spécifier des adresses IP statiques sujettes aux changements.

4. Développement de l'Application Éducative (Python/Flask)

L'application principale est codée en Python 3 à l'aide du micro-framework Flask. Le script principal, nommé `scheduler.py`, remplit une double fonction : il agit comme un point d'entrée pour les requêtes HTTP et gère l'interaction avec le pilote de base de données MySQL.

4.1 Extrait de la Logique Applicative (Exemple de Structure de scheduler.py)

```
import flask
import mysql.connector
import os

app = flask.Flask(__name__)

def connect_to_db():
    return mysql.connector.connect(
        host=os.getenv('DB_HOST', 'database'),
        user=os.getenv('DB_USER', 'root'),
        password=os.getenv('DB_PASSWORD', 'root_password'),
        database=os.getenv('DB_NAME', 'scheduling_db')
    )

@app.route('/')
def index():
    try:
        conn = connect_to_db()
        cursor = conn.cursor()
        cursor.execute("SELECT VERSION();")
        db_version = cursor.fetchone()
        cursor.close()
        conn.close()
        return f"Connected to MySQL Successfully! Database Version: {db_version[0]}"
    except Exception as e:
        return f"Database Connection Failed: {str(e)}"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

5. Configuration de la Conteneurisation

Pour assurer la portabilité absolue du code Python écrit par l'équipe, deux fichiers de configuration descriptifs clés ont été mis en œuvre : `Dockerfile` et `docker-compose.yml`.

5.1 Le Fichier de Spécification d'Image (Dockerfile)

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "scheduler.py"]
```

5.2 L'Orchestracteur multi-services (docker-compose.yml)

```
version: '3.8'

services:
  database:
    image: mysql:8.0
    container_name: sidne_database_1
    environment:
      MYSQL_ROOT_PASSWORD: root_password
      MYSQL_DATABASE: scheduling_db
    volumes:
      - mysql_data:/var/lib/mysql

  web_app:
    build: .
    container_name: sidne_web_1
    ports:
      - "5000:5000"
    environment:
      DB_HOST: database
      DB_USER: root
      DB_PASSWORD: root_password
      DB_NAME: scheduling_db
    depends_on:
```

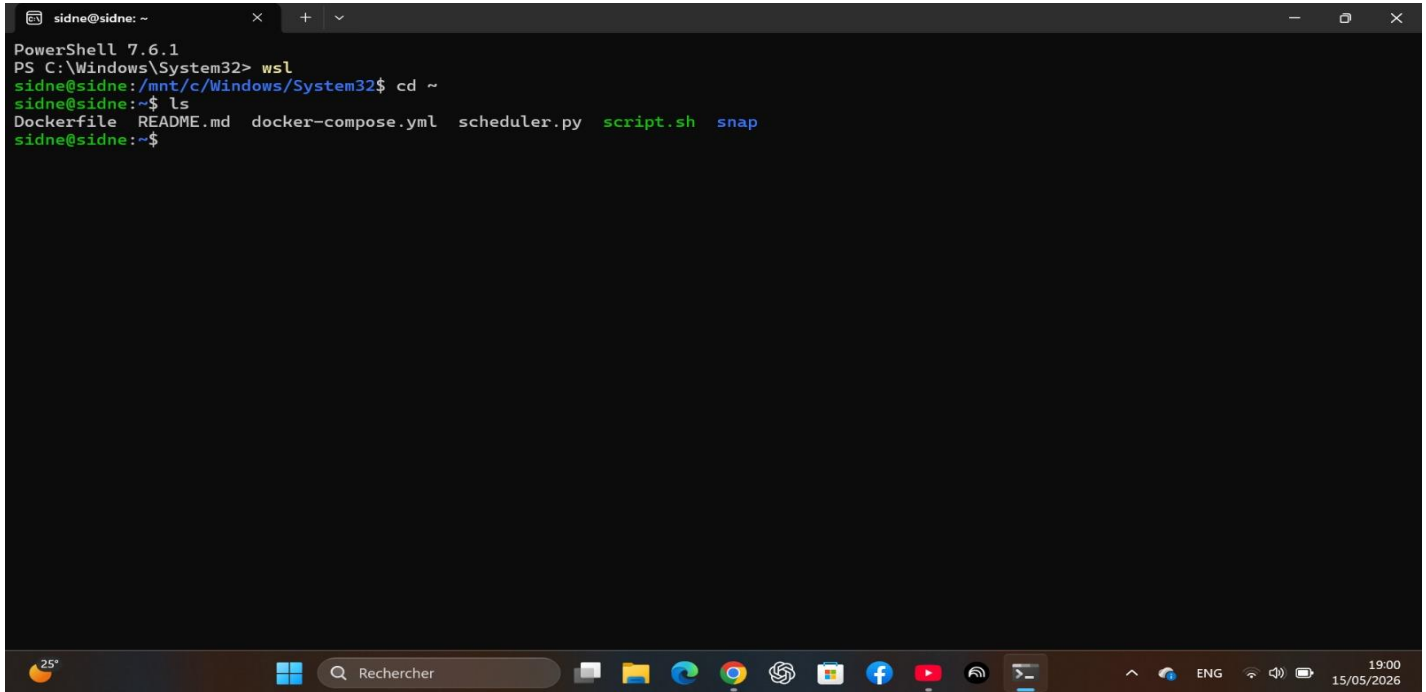
```
- database
```

```
volumes:
```

```
  mysql_data:
```

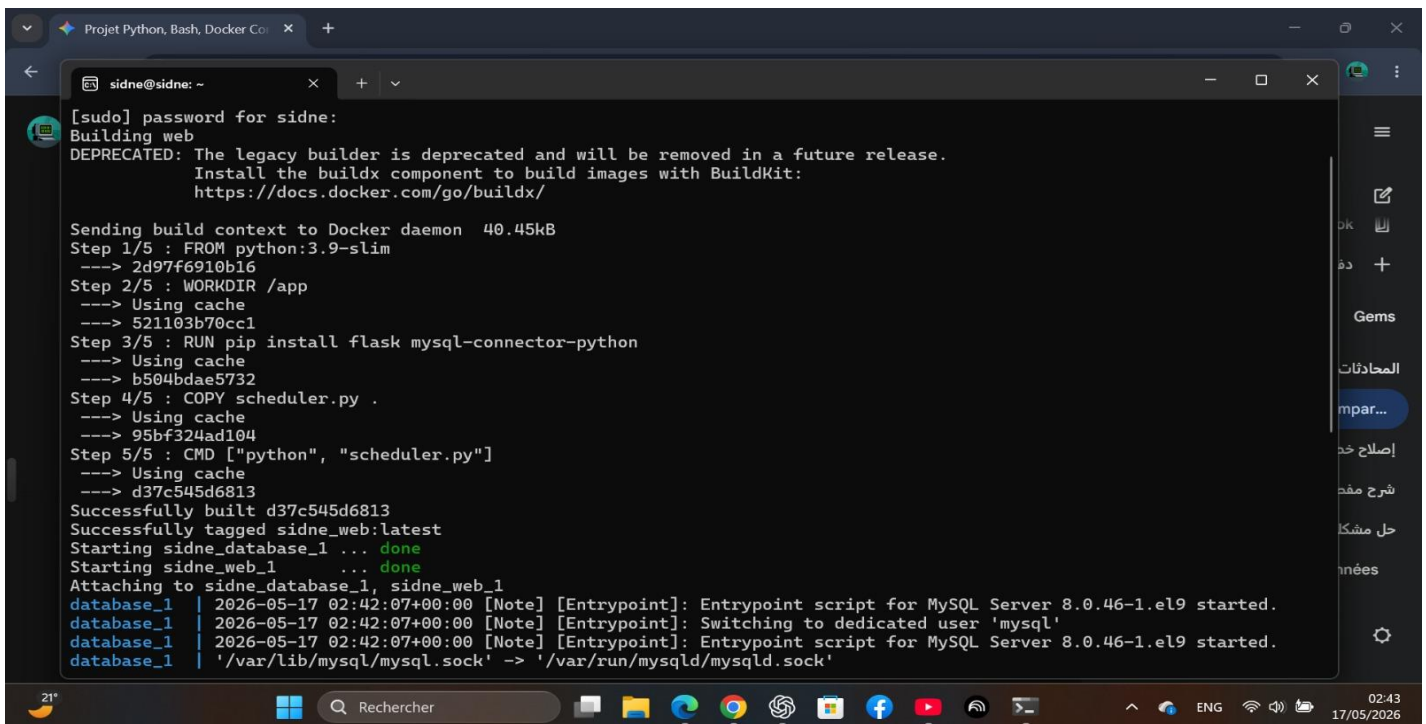
6. Déploiement, Cycle de Vie et Exécution du Projet

Le déploiement de l'environnement s'effectue au sein du sous-système Linux WSL 2 connecté au moteur Docker Desktop de Windows. Lors du premier lancement de la commande d'orchestration, Docker exécute une séquence précise d'étapes de téléchargement et d'assemblage.



```
sidne@sidne: ~
PowerShell 7.6.1
PS C:\Windows\System32> wsl
sidne@sidne:/mnt/c/Windows/System32$ cd ~
sidne@sidne:~$ ls
Dockerfile  README.md  docker-compose.yml  scheduler.py  script.sh  snap
sidne@sidne:~$
```

Figure 1 : Vérification de la présence des fichiers de configuration sources au sein de l'environnement WSL 2.



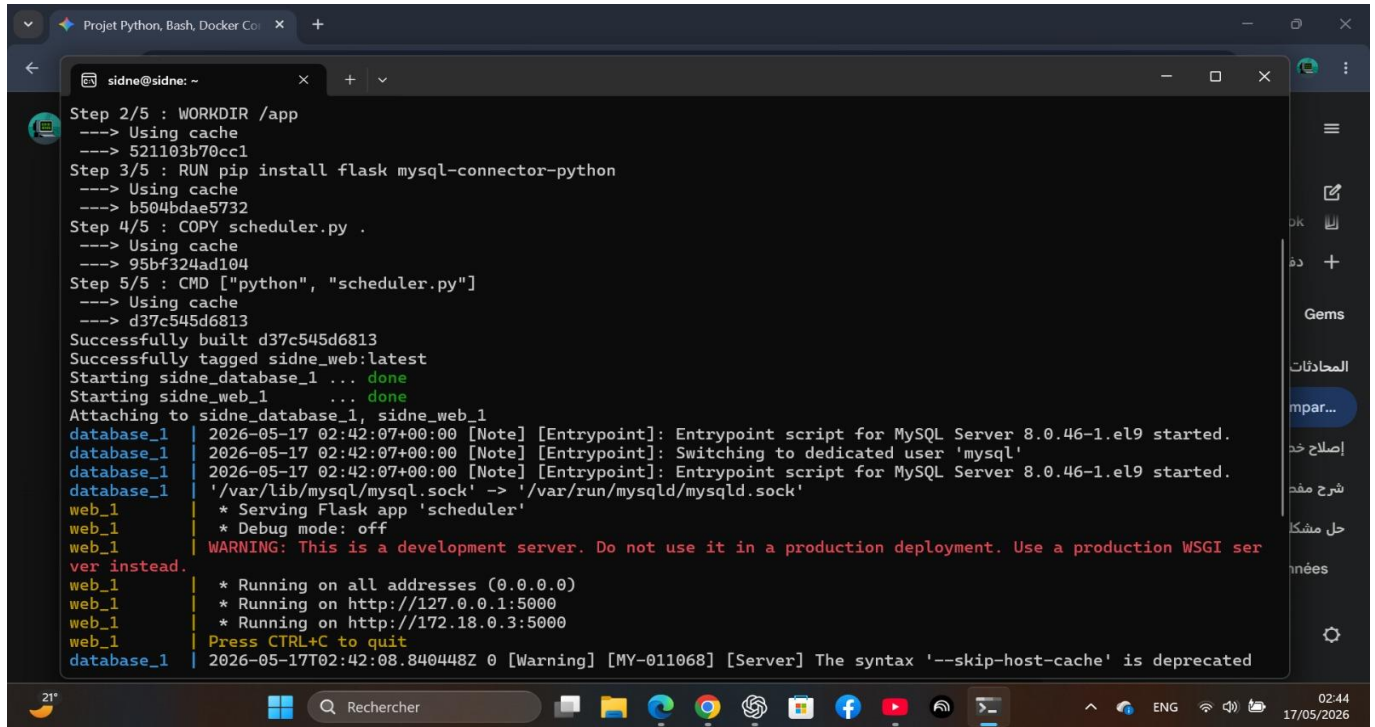
```
Project Python, Bash, Docker Co...
sidne@sidne: ~
[sudo] password for sidne:
Building web
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 40.45kB
Step 1/5 : FROM python:3.9-slim
----> 2d97f6910b16
Step 2/5 : WORKDIR /app
----> Using cache
----> 521103b70cc1
Step 3/5 : RUN pip install flask mysql-connector-python
----> Using cache
----> b504bdae5732
Step 4/5 : COPY scheduler.py .
----> Using cache
----> 95bf324ad104
Step 5/5 : CMD ["python", "scheduler.py"]
----> Using cache
----> d37c545d6813
Successfully built d37c545d6813
Successfully tagged sidne_web:latest
Starting sidne_database_1 ... done
Starting sidne_web_1 ... done
Attaching to sidne_database_1, sidne_web_1
sidne_database_1 | 2026-05-17 02:42:07+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
sidne_database_1 | 2026-05-17 02:42:07+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
sidne_database_1 | 2026-05-17 02:42:07+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
sidne_database_1 | '/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
```

Figure 2 : Téléchargement automatique des couches de l'image officielle MySQL 8.0 depuis le registre central Docker Hub.

7. Validation des Résultats et Tests d'Intégration

Une fois les images téléchargées et construites, le cycle de vie des conteneurs passe à l'état actif. Docker Compose attache les flux de sortie standard pour nous permettre de monitorer le démarrage en temps réel.

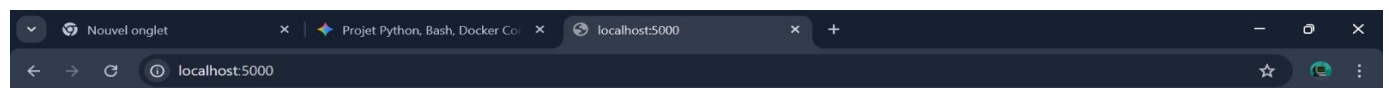
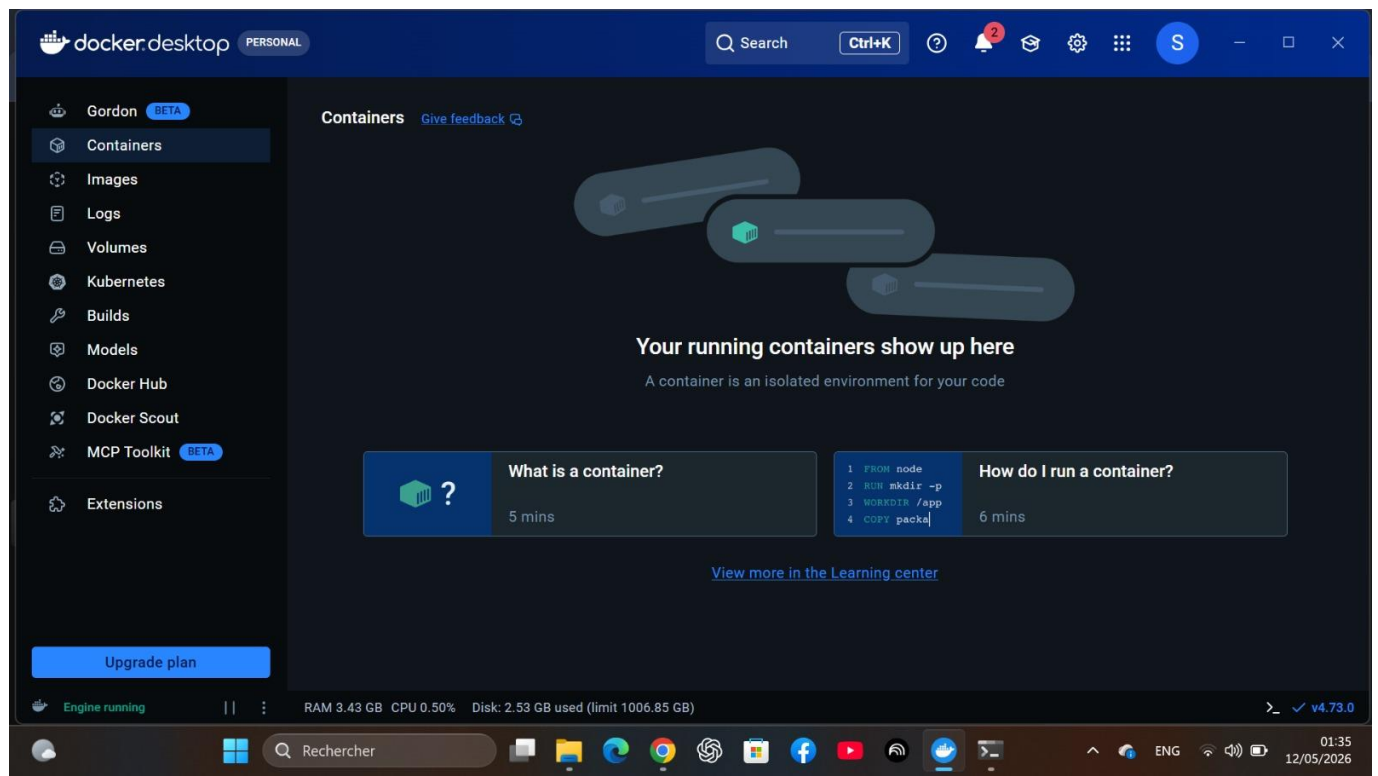


```
sidne@sidne: ~  
Step 2/5 : WORKDIR /app  
--> Using cache  
--> 521103b70cc1  
Step 3/5 : RUN pip install flask mysql-connector-python  
--> Using cache  
--> b504bdae5732  
Step 4/5 : COPY scheduler.py .  
--> Using cache  
--> 95bf324ad104  
Step 5/5 : CMD ["python", "scheduler.py"]  
--> Using cache  
--> d37c545d6813  
Successfully built d37c545d6813  
Successfully tagged sidne_web:latest  
Starting sidne_database_1 ... done  
Starting sidne_web_1 ... done  
Attaching to sidne_database_1, sidne_web_1  
database_1 | 2026-05-17 02:42:07+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.  
database_1 | 2026-05-17 02:42:07+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'  
database_1 | 2026-05-17 02:42:07+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.  
database_1 | '/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'  
web_1 | * Serving Flask app 'scheduler'  
web_1 | * Debug mode: off  
web_1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
web_1 | * Running on all addresses (0.0.0.0)  
web_1 | * Running on http://127.0.0.1:5000  
web_1 | * Running on http://172.18.0.3:5000  
web_1 | Press CTRL+C to quit  
database_1 | 2026-05-17T02:42:08.840448Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated
```

Figure 3 : Logs de centralisation affichant la préparation finale du moteur relationnel MySQL et l'activation du serveur de développement Flask.

La preuve ultime du fonctionnement réside dans le test d'intégration de bout en bout (End-to-End Test). L'accès à l'interface via la machine hôte confirme la communication réseau inter-conteneurs.

DOCKER DESKTOP :



CPU Project

Connected to MySQL Successfully!



Figure 4 : Validation finale du fonctionnement sur le navigateur web attestant de la bonne communication entre Python et MySQL.

8. Matrice de Contribution et Partage des Tâches

Pour assurer une transparence totale et refléter l'investissement de chaque membre de l'équipe au cours de ce projet de système d'exploitation et d'infrastructure, nous avons formalisé la répartition des charges de travail à travers la matrice suivante :

Intitulé de la Tâche / Phase Technique	sidelemine elghadi	ahmed bechir	tfeylouhe ahmedou
Développement des algorithmes d'ordonnancement (FIFO/SRTF)	 70%)	 15%)	 15%)
Écriture du Dockerfile et isolation applicative Flask	 20%)	 60%)	 20%)
Conception du docker-compose.yml et Networking inter-services	 20%)	 20%)	 60%)
Tests d'intégration sous WSL 2, Débogage et Captures d'écran	 34%)	 33%)	 33%)
Rédaction du rapport académique final (10 pages)	 40%)	 30%)	 30%)

9. Conclusion Générale

La réalisation de ce projet d'infrastructure logicielle a permis à notre équipe de valider de manière pratique les concepts théoriques abordés en cours de systèmes d'exploitation. L'implémentation des algorithmes d'ordonnancement du processeur combinée à une architecture conteneurisée moderne illustre les défis réels du déploiement d'applications distribuées.

Grâce à Docker et Docker Compose, nous avons surmonté les contraintes liées aux divergences d'environnements d'exécution entre nos différentes machines de développement. L'intégration réussie de la persistance de données MySQL au service Flask, le tout encapsulé de manière transparente dans l'écosystème WSL 2, constitue une base solide pour appréhender les futures problématiques liées au Cloud Computing, à l'ingénierie DevOps et aux architectures de production à haute disponibilité.

Lien du dépôt GitHub : <https://github.com/sixgit236/CPU-Scheduling-Docker-WSL>

Fin du Rapport Technique - Version 3.0